Modern Programlama Pratikleri

Daha Temiz, Daha Organize ve Daha Güçlü Kod Yazmak



Haftanın Yol Haritası

Bu hafta, profesyonel yazılım geliştirmenin temel taşlarını keşfedeceğiz. Kodunuzu nasıl daha modüler, yönetilebilir ve ekip çalışmasına uygun hale getirebileceğinizi öğreneceksiniz.

01

Fonksiyonlar

Kodun yapı taşlarını anlayalım

02

Modüler Programlama

Kodumuzu nasıl organize ederiz?

03

Versiyon Kontrolü

Neden her mühendisin bilmesi gerekir?

04

Git & GitHub

Versiyon kontrolünün süper kahramanları

05

Ekip Çalışması

Birlikte nasıl kod yazarız?

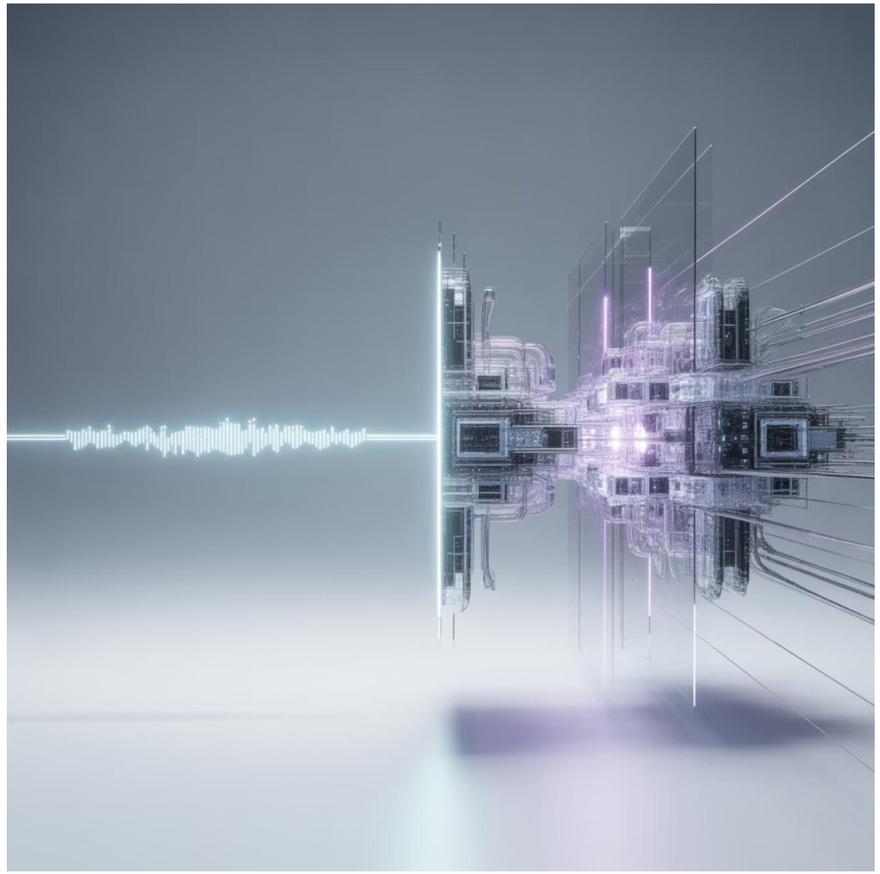
"Hello World"den Gerçek Projelere

Neden "Modern" Pratikler?

İlk başta yazdığımız küçük programlar tek bir dosyada olabilir. Birkaç satırlık bir kod parçasını yönetmek kolaydır ve her şey kontrolünüz altındadır.

Ancak projeler büyüdükçe – yüzlerce, hatta binlerce satır koda ulaştığında – organizasyon ve iş birliği hayati önem kazanır. Kod tabanınızın karmaşıklığı arttıkça, yapılandırılmış yaklaşımlara ihtiyaç duyarsınız.

Modern pratikler, kodun sürdürülebilir, anlaşılır ve ekip çalışmasına uygun olmasını sağlar. Profesyonel yazılım geliştirme, sadece çalışan kod yazmak değil, uzun vadede bakımı yapılabilir kod yazmaktır.





Fonksiyonlar: Tekrar Kullanılabilir Kod Blokları

Fonksiyon, belirli bir işi yapmak üzere tasarlanmış, isimlendirilmiş bir kod parçasıdır. Modern programlamanın temel yapı taşıdır.

"Don't Repeat Yourself"

Aynı kodu tekrar tekrar yazmayın – bir kez yazın, istediğiniz yerde kullanın

Kahve Makinesi Analojisi

Siz sadece düğmesine basarsınız (fonksiyonu çağırırsınız), o sizin için kahveyi yapar

Hazır Araç Kutusu

Her seferinde kahvenin nasıl yapıldığını baştan tarif etmenize gerek kalmaz

Bir Fonksiyon Nasıl Yazılır?

Her fonksiyonun belirli bir anatomisi vardır. Bu yapıyı anlamak, güçlü ve esnek kod yazmanın temelidir.

1

def Anahtar Kelimesi

Fonksiyonu tanımlamak için kullanılan başlangıç komutu

2

Fonksiyon Adı

Fonksiyonunuza verdiğiniz açıklayıcı ve anlamlı isim

3

Parametreler

Fonksiyonun çalışması için dışarıdan aldığı veriler (girdiler)

4

Fonksiyon Gövdesi

İşlemin gerçekleştiği kod bloğu

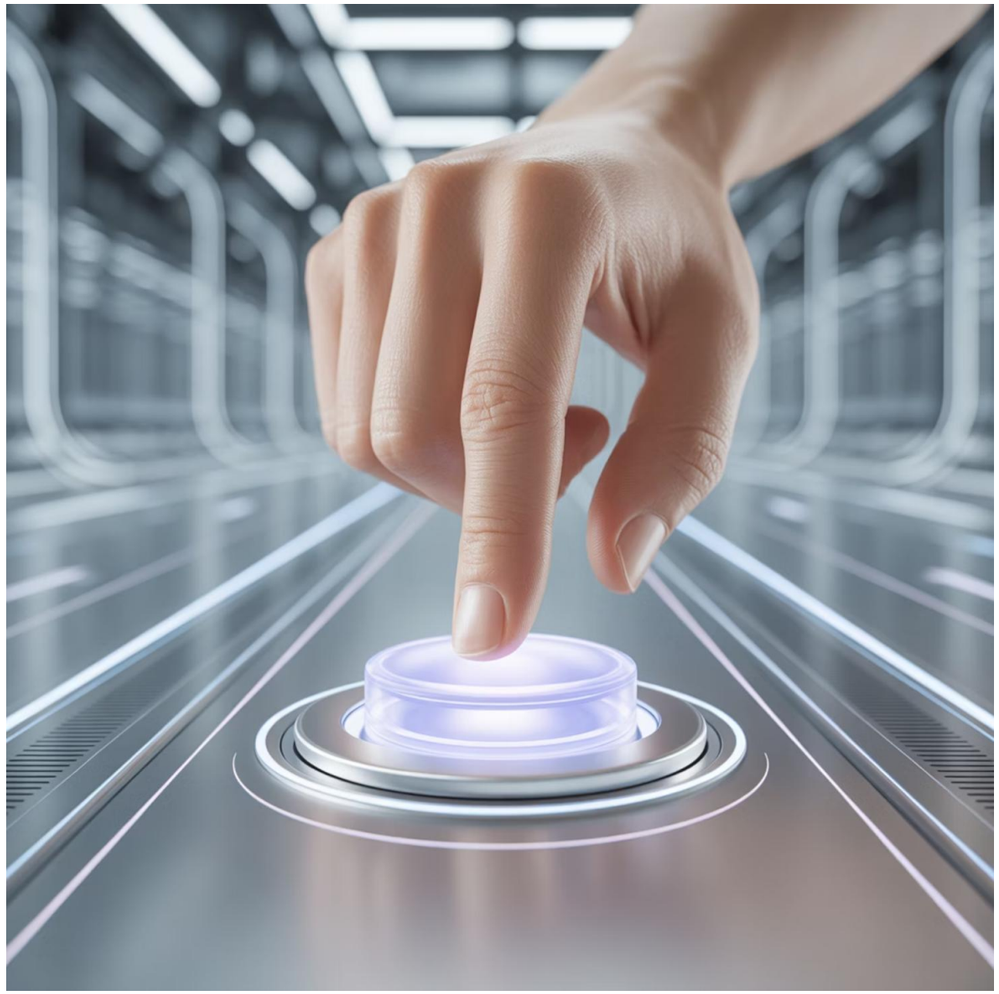
5

return İfadesi

Fonksiyonun yaptığı işlem sonucunda dışarıya döndürdüğü değer (çıktı)

```
def fonksiyon_adi(parametre1, parametre2): # Bu bloğun içindeki kodlar çalışır islem = parametre1 + parametre2 return islem # Fonksiyon bir değer döndürür
```

Kahve Makinesinin Düğmesine Basmak



Fonksiyonu Çağırma ve Kullanma

Fonksiyonu tanımladıktan sonra, onu programınızın istediğiniz yerinde çağırabilir ve kullanabilirsiniz. Bu, fonksiyonların asıl gücünü ortaya çıkarır.

```
# Fonksiyonu tanımlayalımdef topla(sayi1, sayi2):    return sayi1 + sayi2# sayi2# Fonksiyonu çağıralımsonuc = topla(10, 25)print("İşlemin sonucu:", sonuc)# sonucu:", sonuc)# Ekrana 35 yazar
```

Aynı fonksiyonu dilediğiniz kadar, farklı değerlerle çağırabilirsiniz. Her çağrıda yeni bir sonuç elde edersiniz.

Neden Her Yerde Fonksiyon Kullanmalıyız?

Fonksiyonlar, profesyonel yazılım geliştirmenin vazgeçilmez araçlarıdır. İşte size sağladığı temel avantajlar:



Tekrar Kullanılabilirlik

Reusability: Aynı kodu tekrar tekrar yazmaktan kurtarır. Bir kez yazın, her yerde kullanın. Kod tekrarı hem zaman kaybıdır hem de hata riskini artırır.



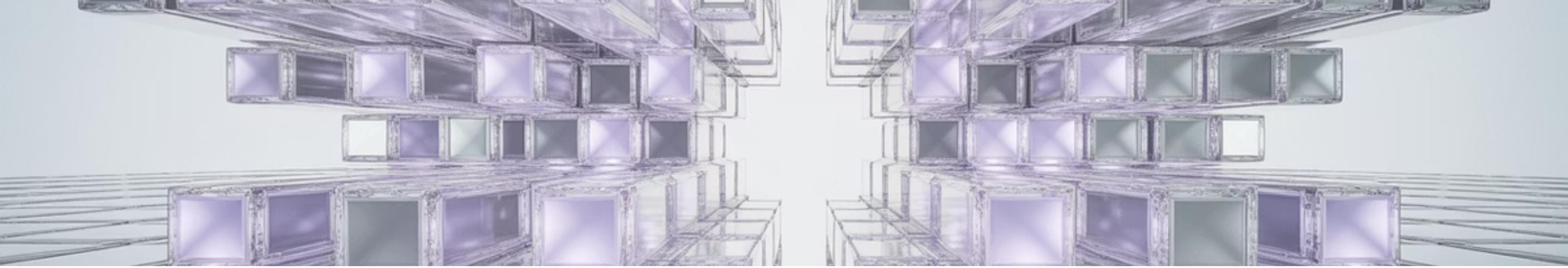
Okunabilirlik

Readability: Karmaşık bir problemi küçük, anlaşılır parçalara böler. `hesapla_kdv()` fonksiyonu, içinde ne yaptığını adıyla belli eder. Kod okuyan herkes ne yaptığını anlar.



Bakım Kolaylığı

Maintainability: Bir hesaplama yöntemini değiştirmeniz gerektiğinde, sadece ilgili fonksiyonu güncellemeniz yeterlidir. Tüm kodda değişiklik yapmaya gerek kalmaz.



Modüler Programlama Nedir?

Projeyi "Böl ve Yönet"

Modüler programlama, büyük bir programı, her biri belirli bir görevi yerine getiren daha küçük, bağımsız ve yönetilebilir parçalara (modüllere) ayırma tekniğidir.

🚗 Araba Analojisi

Bir araba düşünün: Motor, şanzıman, tekerlekler, elektronik sistem... Hepsi ayrı birer modüldür.

🔧 Kolay Bakım

Bir sorun olduğunda tüm arabayı değil, sadece ilgili modülü kontrol edersiniz. Modüler kod da böyle çalışır.

🧩 Bağımsız Parçalar

Her modül kendi işini yapar ve diğer modüllerden bağımsızdır. Bu sayede paralel geliştirme mümkün olur.

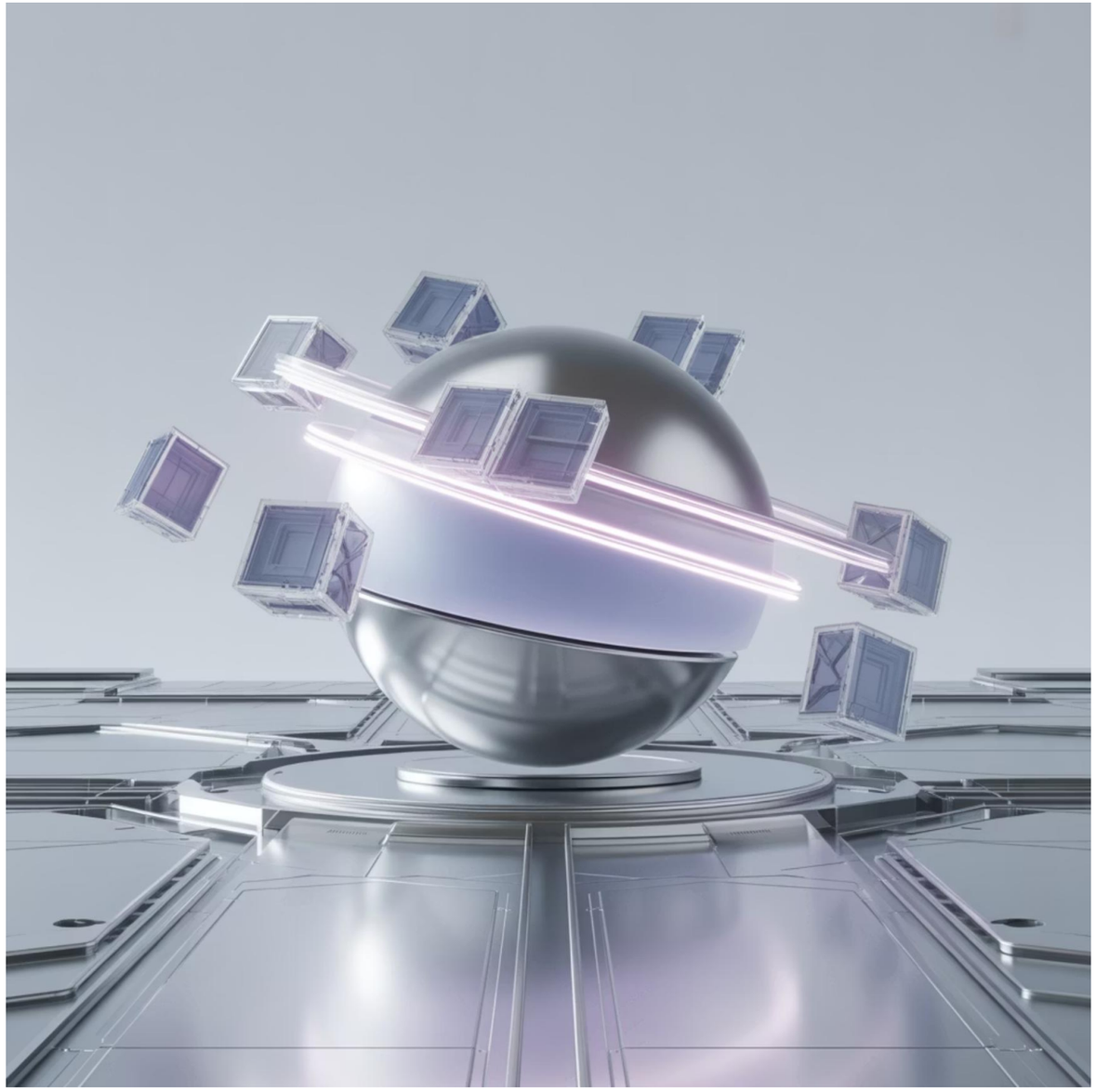
Python'da Modüller: import

Başkalarının Gücünü Kullanmak

Python'da her `.py` dosyası bir modüldür. `import` anahtar kelimesi ile başka dosyalardaki fonksiyonları veya değişkenleri kendi kodumuza dahil edebiliriz.

Python'un zengin standart kütüphanesi, hazır modüllerle doludur. Matematiksel işlemler, tarih/saat işlemleri, dosya okuma/yazma ve daha fazlası için hazır modüller mevcuttur.

```
import math# math modülü içindeki sqrt fonksiyonusayinin_karekoku = math.sqrt(16)print(sayinin_karekoku)  # 4.0# math modülü içindeki pi sabitiprint(math.pi)  # 3.141592...
```





Kendi Modülümüzü Yazmak

Kodumuzu Düzenleyelim

Kendi modüllerinizi oluşturmak, kodunuzu organize etmenin en güçlü yoludur. İşte basit bir örnek:

1

2

hesaplamalar.py

```
def topla(a, b):    return a + bdef carp(a, b):    return a * b
```

ana_program.py

```
import hesaplamalarsonuc1 = hesaplamalar.topla(5, 3)sonuc2 = hesaplamalar.carp(5, 3)print("Toplam:", sonuc1)  # 8print("Çarpım:", sonuc2)  # 15
```

Bu yaklaşımla, matematiksel işlemleriniz bir modülde, veri işleme fonksiyonlarınız başka bir modülde olabilir. Kod tabanınız büyüdükçe, bu organizasyon hayat kurtarıcı olur.



Kodunuz İçin Bir Zaman Makinesi

Versiyon Kontrol Sistemi (VCS) Nedir?

Versiyon kontrol sistemi, bir veya daha fazla dosya üzerinde yapılan değişiklikleri zaman içinde kaydeden ve bu sürümleri daha sonra geri çağırabilmenizi sağlayan bir sistemdir.

😱 Eski Yöntem

- proje_v1.py
- proje_v2.py
- proje_final.py
- proje_asıl_final_bu.py
- proje_bu_sefer_kesin_final.py

✨ Modern Yöntem

Tek bir dosya, sonsuz sürüm. Her değişiklik kaydedilir, istediğiniz zaman geri dönebilirsiniz. Kaos yok, sadece düzen var.

"CTRL+Z"nin Çok Ötesinde

Neden Versiyon Kontrolü Kullanmalıyız?

Versiyon kontrol sistemleri, modern yazılım geliştirmenin can damarıdır. İşte size sunduğu süper güçler:

📝 Değişiklik Takibi

Kim, neyi, ne zaman, neden değiştirdi? Her değişikliğin detaylı kaydını tutar. Kod tarihçeniz kristal berraklığında.

🔙 Hata Düzeltme

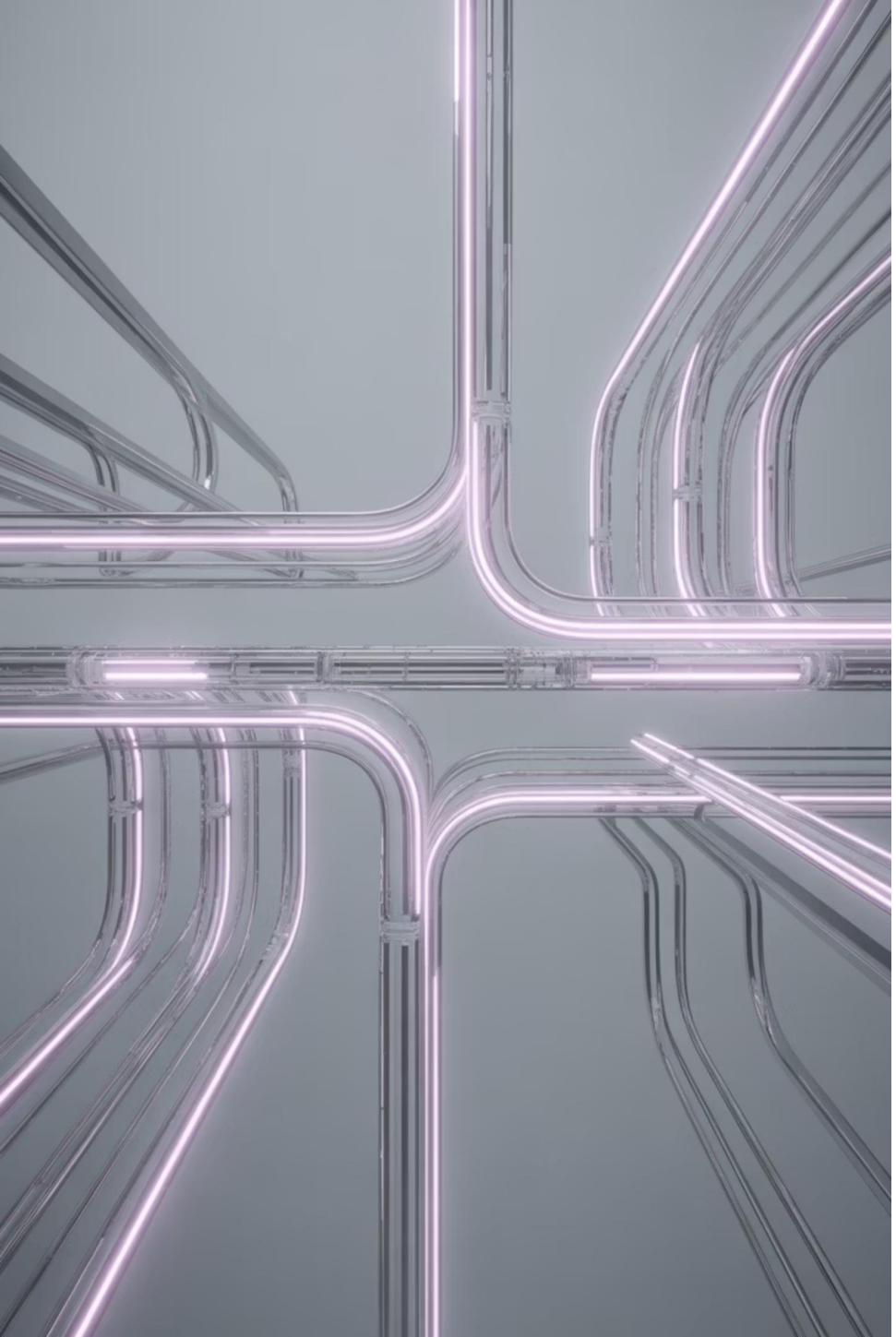
Çalışan bir kod parçasını bozdunuz mu? Panik yapmayın! Tek bir komutla çalışan eski sürüme dönebilirsiniz. Geriye dönük güvenlik ağınız her zaman hazır.

🌿 Paralel Geliştirme

Ana projeyi bozma riski olmadan yeni özellikler deneyebilirsiniz. Branching sayesinde güvenli bir kum havuzunda oynarsınız.

👥 Ekip Çalışması

Birden fazla kişinin aynı proje üzerinde sorunsuzca çalışmasını sağlar. Çatışmalar otomatik tespit edilir ve çözülür.



Tanıştıralım: Git ve GitHub

İkisi Aynı Şey Değil!

Bu iki kavram sıklıkla karıştırılır, ancak aralarında önemli farklar vardır. Her ikisini de anlamak, modern yazılım geliştirme için şarttır.

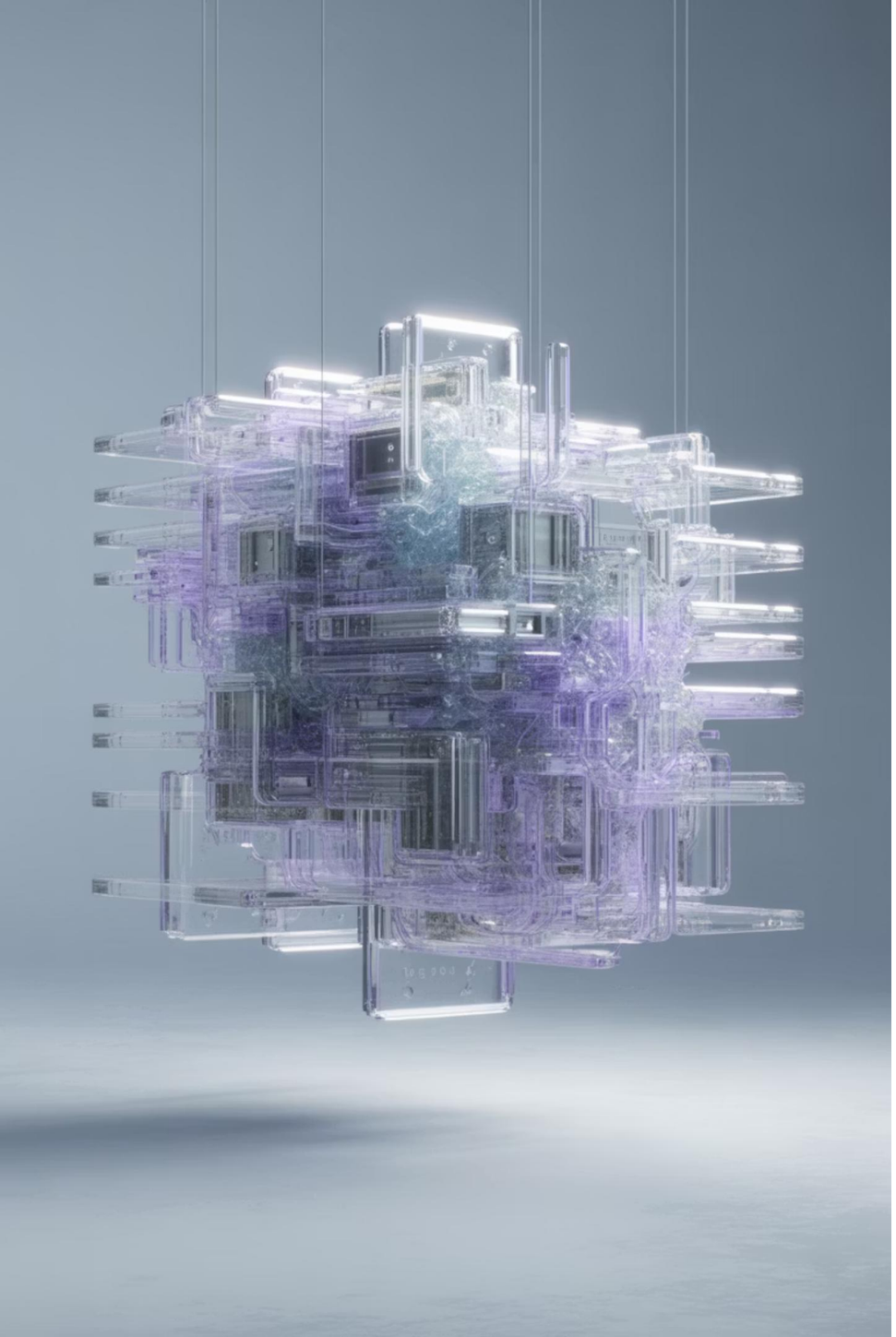
Git

Versiyon kontrolünü yapan **yazılımın kendisidir**. Bilgisayarınızda yerel olarak çalışır. Linus Torvalds tarafından 2005'te geliştirilmiştir.

GitHub

Git ile oluşturduğunuz kod depolarını **internet üzerinde** saklamanızı ve paylaşmanızı sağlayan bir web sitesidir (servistir).

🖼️ Analoji: Git, Microsoft Word programı gibidir; belgelerinizi oluşturur ve değişiklikleri takip edersiniz. GitHub ise Google Drive gibidir; Word belgelerinizi bulutta saklar, başkalarıyla paylaşırsınız ve birlikte düzenlersiniz.



Git'in Alfabesi

Temel Git Kavramları

Git'i etkili kullanabilmek için bu temel kavramları anlamanız gerekir. Bunlar, Git dünyasının yapı taşlarıdır.

Repository (Repo)

Projenizin tüm dosyalarını ve değişiklik geçmişini içeren klasör. Hem yerel bilgisayarınızda hem de GitHub'da bulunabilir.

Commit

Kodunuzun belirli bir andaki anlık görüntüsü, bir "kayıt noktası". Her commit'in açıklayıcı bir mesajı olmalıdır. İyi bir commit mesajı, gelecekteki kendinize armağandır.

Push

Yerel bilgisayarınızdaki (local) commit'leri GitHub'daki sunucuya (remote) gönderme işlemi. Değişikliklerinizi dünyayla paylaşırsınız.

Pull

GitHub'daki en son değişiklikleri kendi bilgisayarınıza çekme işlemi. Ekip arkadaşlarınızın yaptığı güncellemeleri alırsınız.

Günlük Kullanım Döngüsü

Basit Bir Git Akışı

Git kullanımı, tekrar eden basit bir döngüden oluşur. Bu döngüyü bir kez öğrendikten sonra, günlük iş akışınızın doğal bir parçası haline gelir.

1. Değişiklik Yap

Kodunu yaz veya düzenle. Dosyalarınızı dilediğiniz gibi değiştirin.



2. git add

Değişiklikleri "sahneye" ekle. Commit'e dahil edilecek dosyaları seç.

3. git commit

Sahnedeki değişiklikleri açıklayıcı bir mesajla kaydet.

4. git push

Kaydettiğin değişiklikleri GitHub'a yolla. Artık herkes görebilir.

```
# Örnek komutlargit add .git commit -m "Yeni hesaplama fonksiyonu eklendi"git push origin main
```

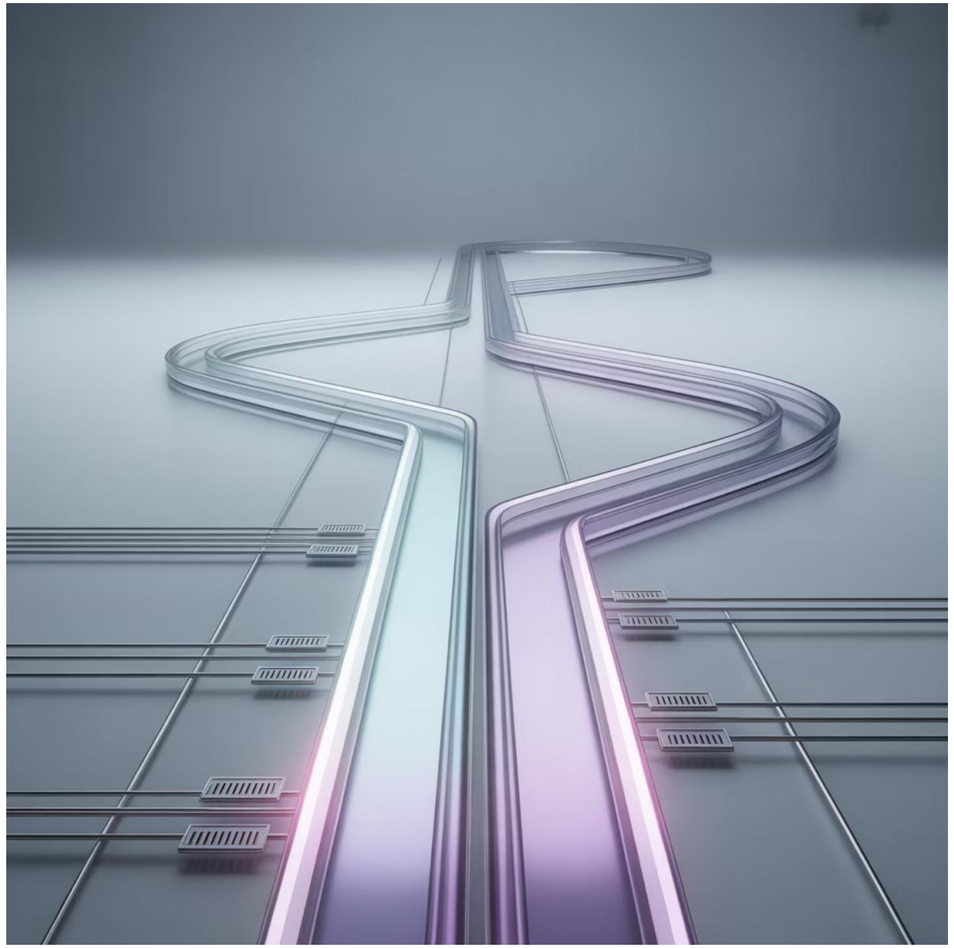
Projeyi Bozmadan Geliştirme

Branch (Dal) ve Merge (Birleştirme)

Branch, projenin ana hattını (main branch) etkilemeden yeni bir özellik veya deneme yapmak için oluşturulan paralel bir çalışma alanıdır.

Bir kitabın ana metnini bozmadan, kenarına notlar alarak yeni bir fikir geliştirmek gibidir. Branch'inizde özgürce deneyler yapabilir, hata yapabilirsiniz. Ana kod güvende kalır.

Merge ise ayrı bir branch'te tamamladığınız işi, projenin ana hattıyla birleştirme işlemidir. Yeni özelliğiniz test edildi ve hazır mı? Zamanı geldi, ana projeye entegre edin!



1. **Ana Dal**
   Çalışan, stabil kod
2. **Yeni Branch**
   Özellik geliştirme
3. **Test & Geliştirme**
   Güvenli ortamda çalışma
4. **Merge**
   Ana dala birleştirme

GitHub Profili: Sizin Dijital CV'niz

Dünyaya Gösterdiğiniz Yüz

İş ve staj başvurularında, birçok şirket artık adayların GitHub profillerini inceliyor. Yaptığınız projeler, katkıda bulunduğunuz açık kaynaklı yazılımlar ve kodlama alışkanlıklarınız sizin en iyi referansınızdır.

Temiz ve Düzenli

Profilinizdeki projeler, README dosyalarıyla iyi dokümante edilmeli. İlk izlenim önemlidir.

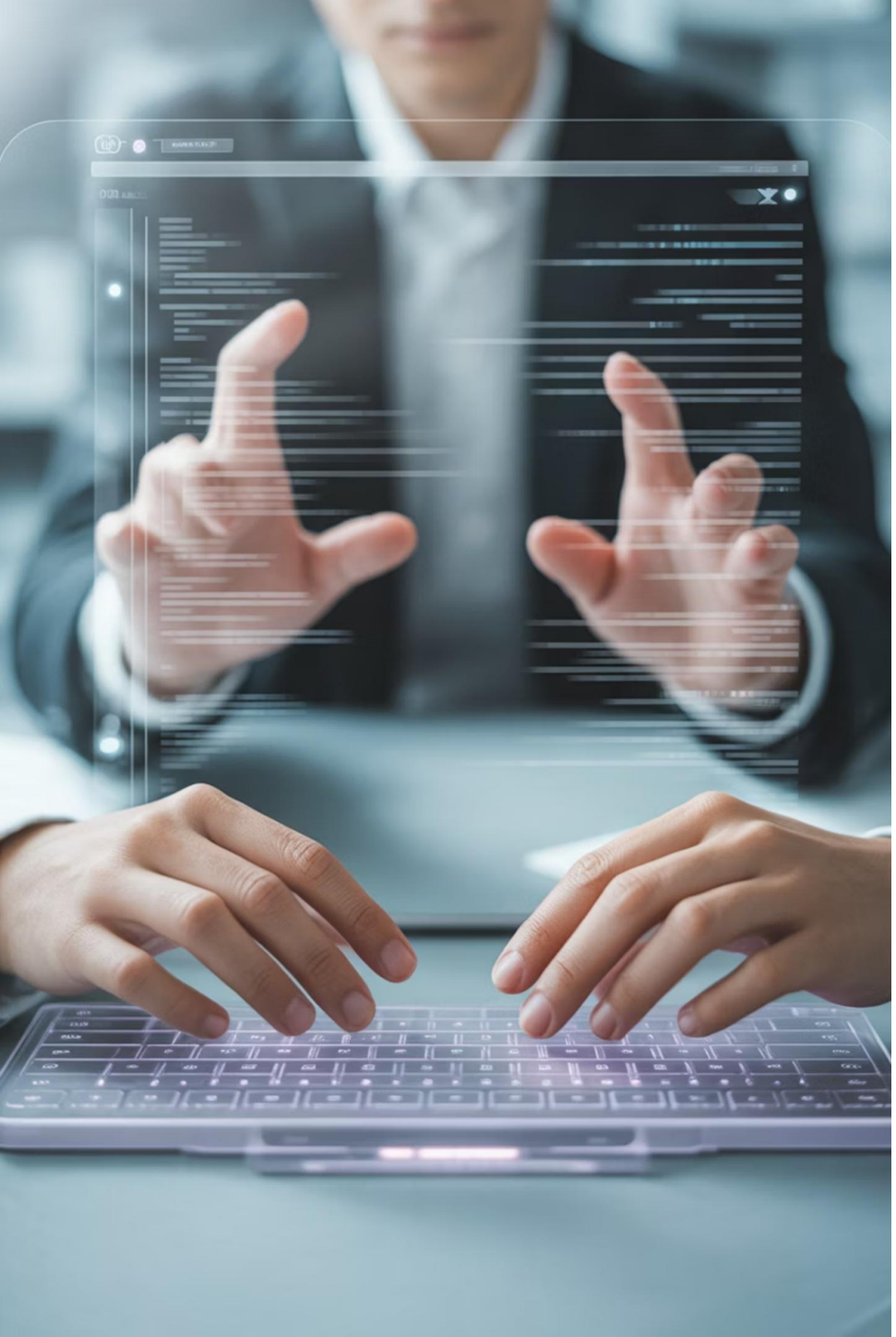
Aktif Katkılar

Düzenli commit'ler, disiplininizi ve tutkunuzu gösterir. Yeşil kareler, çalışkanlığınızın kanıtıdır.

Çeşitli Projeler

Farklı teknolojilerle yapılmış projeler, öğrenme isteğinizi ve yeteneklerinizi sergiler.

💼 Kariyer İpucu: Temiz, düzenli ve aktif bir GitHub profili oluşturmak, kariyeriniz için yapacağınız en iyi yatırımlardan biridir. İşe alım yöneticileri, CV'nizden çok GitHub profilinize bakabilir!



Kollarımızı Sıvayalım!

Haftanın Aktivitesi / Ödevi

1

Bireysel Görev

- GitHub.com'a kaydolun ve profilinizi oluşturun
- Masaüstü için Git'i bilgisayarınıza kurun
- GitHub'da `bilgisayar-muh-giris` adında ilk repository'nizi oluşturun
- Bu repository'e içinde adınızın ve öğrenci numaranızın olduğu bir `README.md` dosyası ekleyip push edin

2

Ekip Çalışması

- 2-3 kişilik ekipler oluşturun
- Bir ekip üyesi bir repository oluştursun ve diğerlerini "collaborator" (işbirlikçi) olarak eklesin
- Ekip olarak, basit bir hesap makinesi programı yazın
- Her üye farklı bir fonksiyonu (toplama, çıkarma, çarpma, bölme) kendi branch'inde yazıp sonra `main` branch'e merge etsin

Bu aktivite, öğrendiğiniz tüm kavramları pratikte uygulama fırsatı sunar. Fonksiyonlar, modüler programlama, Git ve GitHub – hepsi bir arada! Sorularınız olursa, ekip arkadaşlarınızla ve öğretim görevlinizle iletişime geçin. Başarılar!